DESHMUKH TECHNOLOGIES

DTTP · V1.5

Self-Growth Program

DTTP Full-Stack Developer Self-Growth Roadmap

Ten learning levels with timeboxes, worked examples, stop rules, and mentor reviews — turning full-stack reference material into a practical Deshmukh Technologies growth path.

PURPOSE	Self-directed full-stack growth
LEVELS	10 progressive stages
TRACKS	Java / Spring Boot or Python / FastAPI
FRONTEND	React + TypeScript
PRACTICE STACK	SQL + GitHub + Docker + CI/CD + AWS
COMPANION	90-Day DTTP Handbook

Learn. Build. Solve. Review. Deploy. Grow.

INTERNAL TALENT DEVELOPMENT STANDARD

DOCUMENT	VERSION	CLASSIFICATION	OWNER
DTTP Self-Growth Program	1.5	Internal Training Standard	Deshmukh Technologies

ROADMAP

Contents

00 How to use this program

01 Level 1 — Web Fundamentals

02 Level 2 — Frontend Engineering

03 Level 3 — React & Modern Frontend

04 Level 4 — Web Communication & APIs

05 Level 5 — Backend Development

06 Level 6 — Database Engineering

07 Level 7 — Architecture

08 Level 8 — Testing, Security & Performance

09 Level 9 — DevOps & Deployment

10 Level 10 — Professional Software Engineering

11 Incremental Employee Management build

12 Oral review bank

13 Depth ladder

14 Complete reference answers

15 Official progression

00 How to use this program

Full-stack reference material provides a strong foundation. It is **not** the entire DTTP curriculum. Use it as the **Full-Stack Knowledge Reference**, then apply it through Deshmukh Technologies tracks and real projects.

> Do not ask trainees to study the reference as one large syllabus. Complete one level before treating the next as the main focus. Combine each level with Java/Spring Boot or Python/FastAPI, React/TypeScript, SQL, GitHub, Docker, CI/CD, AWS, and real Deshmukh Technologies work.

Document	Role
90-Day DTTP Handbook	Timed training path, project, assessment, and graduation
Self-Growth Program	Deeper knowledge map for study before, during, and after the 90 days
Full-Stack Reference	Topic encyclopedia — not a replacement for practice

STUDY METHOD

1. Read the concept.
2. Explain it in your own words.
3. Build a small example.
4. Break it, then debug it.
5. Commit the work.
6. Ask a mentor to review it.

WEEKLY HABIT

- 20% theory
- 30% exercises
- 40% project work
- 10% review and notes

01 **Web Fundamentals**

02 **Frontend Fundamentals**

03 **React + TypeScript**

04 **HTTP + REST + Web APIs**

05 **Java / Python Backend**

06 **Databases**

07 **Architecture**

08 **Testing + Security + Performance**

09 **Git + Docker + CI/CD + Cloud**

10 **Agile + Professional Engineering**

The destination is a **self-directed full-stack developer** who can build, test, debug, explain, document, and maintain software.

MAPPING TO THE 90-DAY HANDBOOK

Self-growth level	Typical 90-day phase
1–2	Days 1–25 · orientation, programming, web foundation
3–4	Days 26–35 · React and API thinking
5–6	Days 36–55 · backend specialization and database
7	Throughout the project, made explicit in design reviews
8	Days 66–78 · security, testing, debugging
9	Days 79–86 · Docker, CI/CD, AWS
10	Entire program, assessed at the end

A trainee may be ahead or behind the calendar. The level checklist matters more than the day number.

SUGGESTED TIMEBOX

These are focus windows, not deadlines. A trainee who finishes early should deepen the same level with a harder artifact, not skip ahead casually.

Level	Suggested focus	Minimum artifact
1	3–5 days	Request-path diagram + static page
2	7–10 days	Responsive page, form, DOM app, fetch demo
3	8–12 days	Typed React app with layout, routes, and one form
4	4–6 days	API contract + Network notes + Postman collection
5	10–14 days	Authenticated CRUD API on one track
6	7–10 days	Schema, seed data, join report, transaction example
7	3–5 days	Layered architecture diagram and module map
8	7–10 days	Tests, one security write-up, one measured improvement
9	5–8 days	Compose file, CI workflow, deploy notes
10	Continuous	Stories, reviews, daily reports, demo

DAILY NOTEBOOK

1. Topic studied
2. What I can now explain
3. What I built
4. What broke
5. What I still cannot explain
6. Commit hash or PR link
7. Tomorrow's one objective

If there is no notebook entry and no Git artifact, the day does not count as self-growth.

COMMIT STANDARD

```
level-03: add protected employee list route
```

A useful commit is small, named after the level, and reviewable. Do not store a week of mixed work in one commit.

WRITING STANDARD

Notes are part of the work. Write so a mentor can review the day without a meeting. Be specific: name the file, endpoint, status code, or error. Prefer evidence over adjectives. Record what is still unclear.

Field	Sample notebook entry
Topic studied	Protected routes and service layer
What I can now explain	Why employee list state does not belong inside JSX
What I built	`/employees` route, `employeeService.ts`, empty state
What broke	Redirect loop when the token was an empty string
What I still cannot explain	When Context is better than lifting state
Commit / PR	`level-03: add protected employee list route`
Tomorrow	Create-employee form with typed validation

DOCUMENTATION ARTIFACT BY LEVEL

Level	Written artifact the mentor reviews
1	Request-path diagram and URL parts
2	Accessibility and fetch-error notes
3	Component map and where state lives
4	API contract and Network-tab notes
5	Endpoint notes for each implemented route
6	Schema, seed notes, and the SQL behind one ORM call
7	Architecture diagram and three “not yet services” reasons
8	Test list, security note, before/after measurement
9	Compose runbook and CI failure explanation
10	User story, PR walkthrough notes, daily reports

FEATURE NOTE

```
Feature: Create employee
Done when: ADMIN/HR can create; EMPLOYEE cannot; duplicate email is 409
Tests: service duplicate-email; API 401/403/409
Docs: README run steps unchanged; api.md updated
```

DECISION NOTE

```
Decision: keep leave in the same service as employees
Why: one team, one database, one deployable
Not chosen: leave microservice
Revisit when: a second team owns leave
```

01 Level 1 — Web Fundamentals

Purpose: give the trainee a mental model of the web before frameworks.

STUDY

- Client and server roles
- Browser, DNS, domain, IP address, port
- URL: protocol, host, path, query, fragment
- HTTP as request/response
- Static page vs dynamic application
- HTML structure, CSS presentation, JavaScript behavior
- Server-side logic and the meaning of full-stack
- Editor, DevTools, terminal, package manager, Git

MUST BE ABLE TO EXPLAIN

- What happens after a user types a URL
- Difference between frontend and backend
- File server vs application server
- Why JavaScript runs in the browser and Java/Python run on the server

PRACTICE

- Draw browser → DNS → server → response
- Build `index.html`, `styles.css`, `app.js`
- Inspect Elements, Network, and Console
- Serve a static folder locally

- [ ] Can explain client, server, URL, DNS, and HTTP
- [ ] Can build and style a simple page
- [ ] Can inspect HTML, CSS, and network calls
- [ ] Can describe what a full-stack application contains

> A trainee who cannot explain how a browser request reaches a server is not ready for React or Spring Boot.

WORKED EXAMPLE

Explain this URL in writing:

```
https://app.deshmukh.local:5173/employees?status=active#list
```

Identify protocol, host, port, path, query, and fragment. Then open any website, find one document request and one API request in the Network tab, and write the difference.

> Stop rule: do not start Level 2 until the trainee can narrate a request without using the words “it just loads.”

02 Level 2 — Frontend Engineering

Purpose: make the trainee fluent in the browser before React abstractions.

STRUCTURE & STYLE

- Semantic HTML, landmarks, tables, lists
- Forms, labels, input types, native validation
- Cascade, selectors, specificity, inheritance
- Box model, Flexbox, CSS Grid
- Responsive units and media queries

BEHAVIOR & ACCESS

- Variables, types, scope, functions
- Objects, arrays, errors, `try / catch`
- DOM events, create/update/remove nodes
- Fetch: JSON, loading, error states
- Storage, timers, history
- Accessibility: keyboard, contrast, labels, focus

PRACTICE

Responsive landing page | Validated form | DOM todo list | Public JSON API render | Keyboard-only use

- [ ] Can layout a page with Flexbox and Grid
- [ ] Can handle form input and validation
- [ ] Can update the DOM from JavaScript
- [ ] Can fetch JSON and render success/error/loading
- [ ] Can name at least three accessibility checks

WORKED EXAMPLE

Build a “New Employee” HTML form with name, email, department, and joining date. Validate empty fields before submit. After submit, append a row to a table using the DOM. Then fetch a public user list and render names. Make the form usable with Tab and Enter only.

> Stop rule: do not start React until the trainee can do this without a framework.

03 Level 3 — React & Modern Frontend

Purpose: build maintainable UI with **React + TypeScript**.

Components | Props vs state | Effects / lifecycle | Controlled forms | Composition | Context API | Routing | Protected routes | SPA structure | Typed API responses

```
src/
├── components/   pages/   layouts/   services/
├── hooks/        types/   utils/     routes/
└── assets/
```

Practice: multi-page app, login + protected route, form POST, loading/empty/error states, reusable table. Apply this to the Employee Management System frontend.

- [ ] Can create typed React components
- [ ] Can manage local and shared state
- [ ] Can implement routing and a protected page
- [ ] Can integrate an API through a service layer
- [ ] Can keep UI responsive and readable

WORKED EXAMPLE

Build `/login`, protected `/dashboard`, and protected `/employees` with a list plus create form. Put API calls in `services/employeeService.ts`. Types go in `types/employee.ts`. If the token is missing, redirect to login. Show an empty state when the list is `[]`.

> Stop rule: do not start backend specialization until the trainee can explain props, state, effects, and a service layer.

04 Level 4 — Web Communication & APIs

Purpose: teach what actually happens between frontend and backend.

HTTP

- Request/response line, headers, body
- GET, POST, PUT, PATCH, DELETE
- Safe methods and idempotency
- 200, 201, 204, 400, 401, 403, 404, 409, 422, 500
- Cookies, Content-Type, CORS preflight

Realtime

- Short and long polling
- Server-Sent Events
- WebSockets
- When each model is the right choice

API styles

- JSON as a data contract
- REST resources and error payloads
- Pagination and filtering
- GraphQL as a query API
- OpenAPI / Swagger

Both backend tracks implement the same Employee Management API so Java and Python trainees learn the same design.

```
POST /api/auth/login
GET|POST /api/employees    GET|PUT|DELETE /api/employees/{id}
GET|POST /api/departments  GET|POST /api/attendance
GET|POST /api/leaves       PUT /api/leaves/{id}/approve
```

- [ ] Can read a network trace and explain each part
- [ ] Can choose the correct HTTP method and status code
- [ ] Can describe REST resource design
- [ ] Can say when polling, SSE, or WebSockets is appropriate

WORKED EXAMPLE

For `POST /api/employees`, write the expected request JSON, `201` response, `400`/`422` validation response, `401` when the token is missing, and `403` when an EMPLOYEE tries to create another employee. Capture the same calls in Postman.

ERROR PAYLOAD STANDARD

```
{
  "error": "VALIDATION_FAILED",
  "message": "Email is required",
  "field": "email"
}
```

Use one error shape across Java and Python tracks.

> Stop rule: do not start backend specialization until the trainee can choose method, status, and error shape for create, read, update, delete, unauthorized, and forbidden without guessing.

05 Level 5 — Backend Development

Purpose: implement server-side business logic on one DTTP track. Trainees are not required to learn every backend language in the reference.

SHARED BACKEND SKILLS

Validation · DTO / models · Service layer · Repository · Errors & logging · Authn / authz · Environment config

JAVA TRACK

JAVA · SPRING BOOT · REST · JPA / HIBERNATE · SPRING SECURITY · JUNIT / MOCKITO

Controller → Service → Repository → JPA / Hibernate → Database

PYTHON TRACK

PYTHON · FASTAPI · PYDANTIC · SQLALCHEMY · AUTHENTICATION · PYTEST

Router → Service → Repository → SQLAlchemy → Database

Practice: login, employee CRUD, departments, attendance, leave apply/approve, consistent error JSON, one unit test per service.

- [ ] Can implement a validated REST endpoint
- [ ] Can separate controller, service, and repository
- [ ] Can persist and read data through the ORM
- [ ] Can protect at least one endpoint with authentication
- [ ] Can explain a request through every backend layer

WORKED EXAMPLE

Implement `POST /api/employees` on the chosen track. The controller accepts a DTO. The service rejects a missing name or email, rejects a duplicate email, and assigns a valid department. The repository persists the employee.

- Missing email → `400` / `422` with the standard error payload
- Duplicate email → `409`
- Missing token → `401`
- EMPLOYEE caller → `403`
- Success → `201` and the new employee JSON
- One unit test proves the service rejects a duplicate email

Then implement `GET`, `PUT`, and `DELETE` with the same layering.

> Stop rule: if the trainee cannot walk one request through controller → service → repository → database, stay here. Fat controllers do not complete this level.

06 Level 6 — Database Engineering

Purpose: make SQL a first-class skill. **SQL + MySQL / PostgreSQL is mandatory.** NoSQL is introduced later.

RELATIONAL — MANDATORY

- Tables, types, keys, constraints
- One-to-one, one-to-many, many-to-many
- SELECT, INSERT, UPDATE, DELETE
- JOIN, GROUP BY, HAVING, indexes
- Transactions and isolation
- Normalization and query plans
- ORM mapping and N+1 awareness

NON-RELATIONAL — LATER

- Key-value stores
- Document databases
- Graph databases
- Column-oriented databases
- When SQL is still the better default

Practice: schema for employees, departments, attendance, leave; join reports; justified index; all-or-nothing transaction; same query in SQL and ORM.

- [ ] Can design a normalized schema for the training project
- [ ] Can write joins and aggregations by hand
- [ ] Can use transactions correctly
- [ ] Can explain an ORM query in SQL
- [ ] Can name one valid later use of NoSQL

> ORM knowledge must not replace SQL knowledge.

WORKED EXAMPLE

Design and seed this minimum schema:

```
departments(id, name UNIQUE)
employees(id, name, email UNIQUE, department_id FK, joining_date, role)
attendance(id, employee_id FK, work_date, status)
leaves(id, employee_id FK, start_date, end_date, type, status)
```

Then write, by hand: employees with department names (`INNER JOIN`); employees with or without a department (`LEFT JOIN`); leave count by department (`GROUP BY`); and a transaction that approves a leave and writes the related attendance change. If the second write fails, both roll back. Show the SQL the ORM would generate for the leave-count report.

> Stop rule: if the trainee cannot write the join and the transaction without the ORM, stay here.

07 Level 7 — Architecture

Purpose: move from developer thinking to engineer thinking. Study models, but understand **why** architecture changes.

Client-server · Layered / n-tier · Monolith · Modular monolith · SOA · Microservices · Component-based · Microfrontends · Messaging · MVC · MVP · MVVM · Coupling / cohesion

> Do not teach microservices first. Teach Monolith → Layered Architecture → Modular Monolith → Distributed Systems → Microservices.

Practice: draw Employee Management as a layered monolith; identify auth, employees, attendance, leave, reports; list three reasons not to split them into services yet.

Both tracks use the same shape: **React** → **REST** → **Service** → **Persistence** → **MySQL / PostgreSQL**.

- [ ] Can draw the current training system
- [ ] Can name each layer and its responsibility
- [ ] Can explain monolith vs microservices without slogans
- [ ] Can justify the DTTP architecture sequence

WORKED EXAMPLE

Draw one diagram with these layers and one sentence of ownership for each:

```
React pages / services
        ↓
REST controllers or routers
        ↓
Domain services (leave rules, role checks)
        ↓
Repositories / ORM
        ↓
MySQL or PostgreSQL
```

Mark modules: auth, employees, departments, attendance, leave, reports. Write three reasons this training system should remain a layered monolith — for example one database, one team, one deployable, and no independent scaling need yet.

Stop rule: if the trainee says "microservices scale better" without naming a concrete problem the current system has, stay here.

08 Level 8 — Testing, Security & Performance

Purpose: make quality a permanent habit, not a late phase.

Testing

- Unit tests for business rules
- Integration tests for API and database
- TDD as discipline, coverage as signal
- Doubles: dummy, stub, spy, mock, fake
- Frontend component, form, and error-state tests
- Postman collections for the business API

Security

- SQL injection and XSS
- Authentication vs authorization
- Password hashing, JWT risks
- Roles, secrets, environment variables
- TLS / HTTPS, CORS, CSP
- Least privilege and security headers

Performance

- Measure before optimizing
- TTFB, payload size, query time
- Server- and client-side caching
- Images, minification, compression
- Lazy loading and preloading
- Slow SQL and N+1 queries

- [ ] Can write unit and API tests
- [ ] Can explain authn vs authz
- [ ] Can keep secrets out of source control
- [ ] Can measure one performance number and improve it
- [ ] Can name the main project security risks

Never commit passwords, API keys, tokens, private keys, or production credentials to GitHub. Measure performance before optimizing it.

WORKED EXAMPLE

For leave approval, write a service unit test that HR can approve a pending leave, a service unit test that an ordinary employee cannot, and API tests for `401`, `403`, and `409`. Add a short security note: where the JWT is stored, why `localStorage` is risky, and how secrets stay out of Git. Time `GET /api/employees` before and after an index or query change and record both numbers.

Stop rule: if the trainee cannot show a failing test first, or cannot show a measured number, stay here. "It feels faster" does not complete this level.

09 Level 9 — DevOps & Deployment

Purpose: connect code to a running, repeatable delivery path.

Git

- Working tree, staging, commit
- Meaningful messages
- Branch per task, PR, review
- Merge, rebase, conflicts
- Revert and stash

Docker

- Image vs container
- Dockerfile
- Volumes, networks, env vars
- Compose: frontend, backend, database

Deployment

- Build artifacts and hosting
- Container deployment
- Health checks and logs
- Rollback thinking

DTTP DELIVERY PATH

GIT | GITHUB | GITHUB ACTIONS | DOCKER | AWS

Pipeline: Developer → GitHub → Build → Lint → Unit Tests → Integration Tests → Docker Build → Artifact → Deployment → Health Check.

AWS introduction: IAM, EC2, S3, RDS, CloudFront, Route 53, Security Groups, CloudWatch. Understand the purpose of each service before using it.

- [ ] Can complete a branch / PR / review / merge cycle
- [ ] Can run the project with Docker Compose
- [ ] Can describe a CI pipeline
- [ ] Can explain the purpose of the core AWS services
- [ ] Can diagnose a failed deploy from logs

WORKED EXAMPLE

Deliver the Employee Management System as a repeatable path: a feature branch named after the task; Dockerfiles for frontend and backend; a Compose file that starts frontend, backend, and database; a GitHub Actions workflow for lint, unit tests, then image build; a health endpoint after Compose starts; and a one-page deploy note covering what ran, what failed, what the logs said, and how they recovered.

Stop rule: if the trainee cannot start the stack with Compose and explain a failed CI log, stay here. Pushing straight to main / master does not complete this level.

10 Level 10 — Professional Software Engineering

Purpose: move beyond technology into how professional teams work.

AGILE & SCRUM

- Agile principles
- Roles: Product Owner, Scrum Master, Developers
- Events: planning, daily, review, retrospective
- Artifacts: product backlog, sprint backlog, increment
- User stories and acceptance criteria

PROFESSIONAL SKILLS

- Stand-up and written communication
- Requirement analysis and documentation
- Code review: clarity, correctness, security, tests
- Estimation, ownership, asking for help early
- Technical presentation and peer mentoring

Level	Example
Epic	Employee Management
Feature	Leave
User story	As an HR user, I want to approve leave so that attendance stays accurate.
Task	Create leave approve API
Subtasks	Entity, DTO, service rule, controller, test, PR

Daily report: objective, completed work, learning, problems, investigation, resolution, Git activity, next plan.

- [ ] Can write a user story with acceptance criteria
- [ ] Can break work into tasks and subtasks
- [ ] Can review code with specific comments
- [ ] Can present a feature to a mentor
- [ ] Can describe their own skill gaps honestly

WORKED EXAMPLE

Write this story and the work under it:

As an HR user, I want to approve or reject a pending leave request so that attendance stays accurate.

- Only HR or ADMIN can approve or reject.
- A pending leave can be decided once.
- The employee can see the new status.
- A rejected request does not change attendance.
- Tests, review, and notes exist before the story is called done.

Break it into backend, frontend, test, and documentation tasks. Present a 10-minute pull-request walkthrough. Keep daily reports for ten consecutive training days.

Stop rule: if the trainee cannot state acceptance criteria without saying "it works," stay here.

11 Incremental Employee Management build

The same project grows through the ten levels. Do not wait until Level 5 to start it, and do not rebuild a new app at every level.

Level	What to add to the same Employee Management System
1	Static employee table page and a request-path diagram
2	New-employee form, DOM table, public fetch demo
3	React login, protected list, create form, service layer, mocked API
4	Written API contract, error shape, Postman collection
5	Real authenticated CRUD on the chosen backend track
6	Schema, seed data, join report, leave transaction
7	Layered diagram and module map attached to the repo
8	Service tests, API tests, security note, one measured query
9	Compose file, CI workflow, health check, deploy notes
10	Stories, PR walkthrough, daily reports, mentor demo

ROLE MATRIX THE TRAINEE MUST IMPLEMENT

Action	ADMIN	HR	EMPLOYEE
Manage users and roles	Yes	No	No
Create / update employees	Yes	Yes	No
View own profile	Yes	Yes	Yes
Record / view attendance	Yes	Yes	Own only
Apply for leave	Yes	Yes	Yes
Approve / reject leave	Yes	Yes	No

12 Oral review bank

A mentor can close a level with these questions. The trainee answers without reading notes. A vague answer means the level is not closed.

Level	Ask this
1	What happens after a user types a URL? What is the difference between a file server and an application server?
2	Why did this CSS rule not apply? How is `fetch` different from reloading the page?
3	Where should employee-list state live? Why is the API call not inside the JSX?
4	When is the answer `401` instead of `403`? What JSON do you return for a missing email?
5	Which layer owns “email must be unique”? What does the controller return if the service throws a conflict?
6	Write the SQL for employees with department names. What does a rollback protect in leave approval?
7	Why is this system still a monolith? What would have to be true before leave becomes its own service?
8	What did you measure? What test proves an employee cannot approve leave?
9	Why did CI fail? What is the difference between an image and a container?
10	What is the acceptance criteria for leave approval? What would you tell a peer in a review?

13 Depth ladder

Use this when a trainee finishes a level early. Do not skip ahead. Deepen the same level.

Level	Minimum	Expected	Stretch
1	Static page + URL parts	Request-path diagram + DevTools notes	Compare static hosting vs an application server
2	Form + table	Keyboard-only use + fetch error state	Accessible date field and validation messages
3	Login + one list	Protected routes + service layer	Empty, loading, and error states on every page
4	Method/status table	Postman collection + error contract	Pagination and filter query design
5	One CRUD resource	Authn + one role check + one service test	Attendance and leave on the same layering
6	Schema + one join	Report query + transaction	Explain an EXPLAIN / query plan
7	Layer diagram	Module map + three “not yet services” reasons	Sketch a later modular-monolith split
8	Two tests	401/403/409 tests + security note	Before/after timing on one query
9	Branch + PR	Compose + failing-then-passing CI	Health check and rollback note
10	One user story	Ten daily reports + PR walkthrough	Mentor a peer on one topic

14 Complete reference answers

These are passing answers and contracts. A trainee who cannot produce something this specific is not finished, even if the checklist is ticked.

MODEL ORAL ANSWERS

Level	A passing answer sounds like this
1	The browser reads the URL, asks DNS for the host, connects to the port, sends HTTP for the path, and renders the response. A file server returns `index.html`. An application server runs Java or Python and may read a database first.
2	`fetch` leaves the page in place. A submit without `preventDefault` reloads. If CSS fails, check selector, specificity, inheritance, and later overrides. Keyboard use means Tab plus Enter or Space.
3	List data is fetched in a service, stored in page state, and passed as props. The token lives in auth context. Missing token redirects to `/login`. JSX does not contain `fetch` URLs.
4	`401` means the server does not know the caller. `403` means it knows and refuses. Create is `POST` and success is `201`.
5	Unique email lives in the service and a database constraint. The controller only maps the result to HTTP. A conflict is `409`, not `500`.
6	`INNER JOIN` needs a department. `LEFT JOIN` keeps employees without one. A leave transaction commits both writes or rolls both back.
7	This system stays a layered monolith because one team, one database, and one deployable are enough. Microservices add cost before there is an independent scale problem.
8	A unit test must fail when an EMPLOYEE approves leave. Coverage is not proof. A performance claim needs two measured numbers.
9	An image is the packaged filesystem. A container is a running instance. CI runs the same tests and blocks merge when they fail.
10	Acceptance criteria are testable: who can act, what status changes, and what must not happen. “It works” is not a criterion.

LOGIN CONTRACT

```
POST /api/auth/login
{ "email": "hr@deshmukh.local", "password": "correct-password" }

200 { "token": "<jwt>", "role": "HR", "employeeId": 12, "name": "Asha Patil" }
401 { "error": "INVALID_CREDENTIALS", "message": "Email or password is wrong" }
```

CREATE EMPLOYEE CONTRACT

```
POST /api/employees
Authorization: Bearer <hr-or-admin-token>
{ "name": "Rohan Deshmukh", "email": "rohan@deshmukh.local",
  "departmentId": 3, "joiningDate": "2026-08-18", "role": "EMPLOYEE" }

201 { "id": 41, "name": "Rohan Deshmukh", "email": "rohan@deshmukh.local",
      "departmentId": 3, "departmentName": "Engineering",
      "joiningDate": "2026-08-18", "role": "EMPLOYEE" }
```

Case	Status	Error code
Missing email	400 / 422	`VALIDATION_FAILED`
No token	401	`UNAUTHENTICATED`
EMPLOYEE caller	403	`FORBIDDEN`
Email already used	409	`CONFLICT`
Department missing	404	`NOT_FOUND`

LEAVE APPROVE CONTRACT

```
PUT /api/leaves/18/approve
Authorization: Bearer <hr-token>
{ "decision": "APPROVED" }

200 { "id": 18, "employeeId": 41, "status": "APPROVED", "decisionBy": 12 }
409 { "error": "CONFLICT", "message": "Leave is already APPROVED" }
```

Narrate: apply (`PENDING`) → HR approves (`APPROVED`) → second approve (`409`) → employee sees the new status. Reject uses `"REJECTED"` and must not write attendance.

SQL THE TRAINEE MUST WRITE

```
SELECT e.id, e.name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.id = e.department_id
WHERE e.role = 'EMPLOYEE';

SELECT d.name, COUNT(l.id) AS leave_count
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
LEFT JOIN leaves l ON l.employee_id = e.id AND l.status = 'APPROVED'
GROUP BY d.name;
```

WRONG ANSWERS THAT FAIL THE LEVEL

Level	Fails if the trainee says
1	“The browser just loads it.”
2	“I used a div for the form because it was easier.”
3	“I called fetch inside the button JSX.”
4	“401 and 403 are the same.”
5	“The controller checks the duplicate email and talks to the table.”
6	“The ORM writes the join. I do not need SQL.”
7	“We should start with microservices so it scales.”
8	“I did not write a failing test. Coverage is 90%.”
9	“I pushed to master because the change was small.”
10	“Done means it worked on my laptop.”

15 Official progression

Level	Focus	Outcome
1	Web Fundamentals	Can explain how a browser request becomes a response
2	Frontend Engineering	Can build accessible pages and DOM interactions
3	React + TypeScript	Can build a structured SPA with routing and forms
4	HTTP + REST + Web APIs	Can reason about frontend–backend traffic
5	Java or Python backend	Can implement a secure REST API
6	Databases	Can design and query SQL independently
7	Architecture	Can choose structure for a reason
8	Testing, security, performance	Can protect, verify, and measure software
9	Git, Docker, CI/CD, cloud	Can deliver a running system
10	Agile + professional engineering	Can work as a team engineer

MENTOR CHECKPOINTS AND COMMON MISTAKES

Level	Mentor asks	Common mistake	Required artifact
1	What happens after a URL is entered?	Jumping to React immediately	Request-path diagram + static page
2	Why did this CSS rule not apply?	Ignoring accessibility and errors	Responsive page + fetch demo
3	Where does this state belong?	Putting API calls inside JSX	Typed React feature with routes
4	Why is this 403, not 401?	Treating all failures as 500	Network notes + API contract
5	Which layer owns this rule?	Fat controllers, no tests	CRUD API with one service test
6	Show the SQL behind this ORM call	Using only generated queries	Schema + join report query
7	Why is this still a monolith?	Copying microservices slogans	Architecture diagram
8	What did you measure?	Optimizing before measuring	Tests + one security note
9	Why did CI fail?	Committing to main directly	Compose file + Actions workflow
10	What is the acceptance criteria?	“It works on my machine”	Story, PR walkthrough, daily reports

A level is not complete because the trainee attended a session. It is complete when the artifact exists in Git, the trainee can explain it, and a mentor has reviewed it.

> The reference material is the knowledge map. Deshmukh Technologies projects are the proof. A trainee completes this program when they can independently build, test, debug, explain, document, and maintain a full-stack application.

DTTP — DESHMUKH TECHNOLOGIES TRAINEE PROGRAM

Self-Directed Full-Stack Developer

Ten levels. One practice stack. Real projects.

Learn. Build. Solve. Review. Deploy. Grow.